

SERIE DE TP #01

Thème : Programmation sous Office via les Macros VBA

Exercice #01 : « Modèle & Formulaire Utilisateur »

Objectifs de la manipulation

- Se familiariser avec l'environnement de développement ;
- Découvrir la notion de projet et ses composantes ;
- Créer un projet de Mini-Calculatrice à travers lequel vous manipulerez un nombre important de fonctionnalités.



On voudrait réaliser un formulaire utilisateur sous Word qui permettra de saisir des fiches de note de contrôle continu puis de les insérer dans un tableau Word qui se trouve dans un document créé sur la base d'un modèle « dotm ».

Nous allons le faire à travers les étapes suivantes :

1. Créer un modèle de document Word que vous enregistrez sous le nom « ModèleNotesCC-Gr<Num de votre groupe>.dotm ». Le but est d'éviter de travailler avec le modèle « Normal.dot » ou « Normal.dotm ».
2. Créer ensuite un document word sur la base du modèle de la question 1 et que vous enregistrerez sous le nom « NotesCC-Gr<Num de votre groupe>.docm ».
3. Dans ce document, placer le contenu suivant :

Liste des Notes de Contrôle Continu

Matricule	Nom	Prénom	Note CC1	Note CC2

4. A partir du ruban « Développeur », placer en bas du tableau un bouton dont le rôle sera d'afficher le formulaire qui devra permettre d'insérer les informations dans le tableau de la question 3. Vous nommerez ce bouton « AjoutLigne » et vous lui changerez son titre vers « Fiche Etudiant » par la propriété « caption ».
5. Basculer vers le visual basic puis créer un formulaire contenant les mêmes champs (avec les mêmes noms) que le tableau de la question 3. Ajouter aussi deux boutons « Ajouter » et « Fermer » sur le formulaire. Voici à quoi doit ressembler le formulaire :

Par la table des propriétés, nommer le formulaire « FicheEtudiant »¹.

6. Dans l'événement « click » du bouton « AjoutLigne » de la question 4 taper dedans l'instruction permettant d'afficher le formulaire. L'instruction est : `FicheEtudiant.show`

Tester le bouton. Que remarquez-vous ? Essayer en suite l'instruction `FicheEtudiant.show(0)`. Qu'en déduisez-vous ?

7. Dans l'événement « click » du bouton « Annuler », placer l'instruction `FicheEtudiant.Hide` puis tester sur le document Word. Qu'en déduisez-vous ?
8. De même, pour le bouton titré « Ajouter »², réaliser les contrôles de chaque champ en interceptant l'événement « EXIT » à l'aide de l'instruction `If ... End If` afin de s'assurer que les champs de type texte ne sont pas vides ou numériques et que les champs des notes sont bien dans l'intervalle [0 ... 20]. En cas de présence d'erreur, afficher un message d'avertissement à l'aide de l'instruction `MsgBox`. Voici deux exemples de code à écrire pour intercepter l'événement EXIT du champ Matricule (chaîne numérique) et du champ Nom (chaîne alphabétique) :

```
Private Sub Matricule_Exit(ByVal Cancel As MSForms.ReturnBoolean)
'
' Contrôler Matricule
'
Matricule.Text = Trim(Matricule.Text)
If Matricule.Text = "" Then 'Matricule vide
    MsgBox "Matricule VIDE !!", vbExclamation, "Erreur !"
    Cancel = True
ElseIf Not IsNumeric(Matricule.Text) Then
    MsgBox "Matricule non NUMERIQUE !!", vbExclamation, "Erreur !"
    Cancel = True
Else
    Nom.SetFocus
End If
End Sub
```

Nom

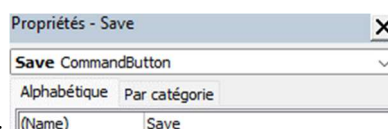
exit

```
Private Sub Nom_exit(ByVal Cancel As MSForms.ReturnBoolean)
'
' Contrôler Nom
'
Nom.Text = Trim(Nom.Text)
If Nom.Text = "" Then 'Nom vide
    MsgBox "Nom VIDE !!", vbExclamation, "Erreur !"
    Cancel = True
ElseIf Not Nom.Text Like "[A-Za-z]*" Then
    MsgBox "Nom non ALPHABETIQUE !!", vbExclamation, "Erreur !"
    Cancel = True
Else
    Prenom.SetFocus
End If
End Sub
```

Lorsque tout est correct, vous insérerez le contenu de la fiche dans une ligne du tableau en utilisant les instructions suivantes :

<code>Selection.TypeText <nom d'un champs ></code>	//Permet de placer un contenu dans une cellule
<code>Selection.MoveRight</code>	//Aller vers la cellule suivante (à droite)

¹ Le tout collé, i-e, pas d'espace entre les mots.



² La variable associée à ce bouton se nomme save :

Après avoir discuté les avantages et inconvénients de cette solution en séance de TP, nous allons procéder à une seconde approche de programmation du bouton « Ajouter » en manipulant la structure de la table « Liste des Notes » elle-même par programmation, ce qui nous permettra de maîtriser les ajouts dans le tableau et non n'importe où dans le document.

Dans cette seconde solution, la table des notes est traitée comme une matrice de plusieurs lignes et 5 colonnes où il s'agira d'insérer chaque champ de la fiche dans la cellule lui correspondant dans la prochaine ligne vide disponible.

9. Pour le faire, nous allons mettre en commentaire les lignes écrites précédemment puis taper les lignes actives telles que vous le montre cette capture d'écran :

```
Private Sub Save_Click()
'-----
' Première Solution
'-----
' Ajouter les valeurs des champs dans la table
'Selection.TypeText Text:=Matricule.Text
'Selection.MoveRight
'Selection.TypeText Text:=Nom.Text
'Selection.MoveRight
'Selection.typeText Text:=Prenom.Text
'Selection.MoveRight
'Selection.typeText Text:=NoteCC1.Text
'Selection.MoveRight
'Selection.typeText Text:=NoteCC2.Text
'-----
' Seconde Solution
'-----
' Déclarer les variables nécessaires
Dim MyDocument As Document ' pointeur vers document
Dim MyTable As Table
Dim NumLigne As Integer

' Initialiser les variables
Set MyDocument = ThisDocument
Set MyTable = MyDocument.Tables(1)

' Trouver la prochaine ligne vide dans la table
NumLigne = MyTable.Rows.Count
If Trim(MyTable.Cell(NumLigne, 1).Range.Text) <> vbCr Then
    MyTable.Rows.Add
    NumLigne = NumLigne + 1
End If

' Transférer les données depuis la fiche vers la table
MyTable.Cell(NumLigne, 1).Range.Text = Matricule.Value
MyTable.Cell(NumLigne, 2).Range.Text = Nom.Value
MyTable.Cell(NumLigne, 3).Range.Text = Prenom.Value
MyTable.Cell(NumLigne, 4).Range.Text = NoteCC1.Value
MyTable.Cell(NumLigne, 5).Range.Text = NoteCC2.Value

' Vider les champs du formulaire après chaque Ajout
Matricule.Text = ""
Nom.Text = ""
Prenom.Text = ""
NoteCC1.Text = ""
NoteCC2.Text = ""
Matricule.SetFocus

End Sub
```

Pour mettre en œuvre la troisième solution, nous allons créer une copie du document précédents que l'on nommera « NotesCC-G<NumGroupe>_Bis.docm » afin de ne pas encombrer le document avec toutes les solutions.

La troisième solution : la seconde solution semble nettement plus élégante que la première puisqu'elle a permis de manipuler la table word comme un objet VBA de type table. Dans cette nouvelle solution, on voudrait expérimenter l'utilisation d'une feuille Excel que l'on structurera exactement comme la table word et dans laquelle nous mettrons les données relatives aux notes CC des étudiants.

Ensuite on ouvrira la feuille de calcul nommée « DataGX » appartenant au classeur Excel « DataNotesCC_GX.xlsx » dans l'espace Visual Basic de Word afin de pouvoir lire les données puis les injecter dans la table Word.

10. Dans le document word, prendre le soin de supprimer toutes les lignes de la table « Liste des notes CC » puis ajouter le bouton « Importer de puis Excel » comme l'illustre la figure suivante :

Liste des Notes de Contrôle Continu

Matricule	Nom	Prénom	Note CC1	Note CC2
-----------	-----	--------	----------	----------

Ajouter Ligne ...	Importer Depuis Excel
-------------------	-----------------------

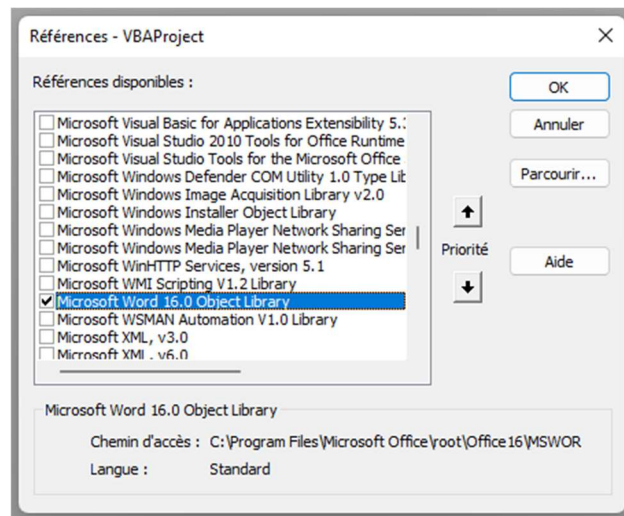
11. Créer un fichier Excel que vous nommerez « DataNotesCC_GX.xlsx » puis saisir les données dans une feuille de calcul que vous nommerez « DataGX » comme le montre la capture suivante :

	A	B	C	D	E
1	Matricule	Nom	Prénom	Note CC1	Note CC2
2	212100031515	ABDI	Yousra	10	12
3	222200031656	AYYAL	Hassan	13	12.75
4	202000034278	SELLAL	Ahsen	14.5	16
5	212100032623	DJABI	Islam	11	11
6	242400032387	DOUIB	Omar	7.5	11.75
7	222200032425	AYOUBI	Ihssaan	12.5	14
8	252500034568	DELLAL	Hafidha	8	7.25
9	202000034545	HENDOUCI	Nora	13	12
10	222200035689	BELKACEMI	Mourad	10	10
11					
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					
22					
23					

12. A travers l'onglet « DEVELOPPER » de l'excel, aller vers « Visual Basic » puis vers le menu :

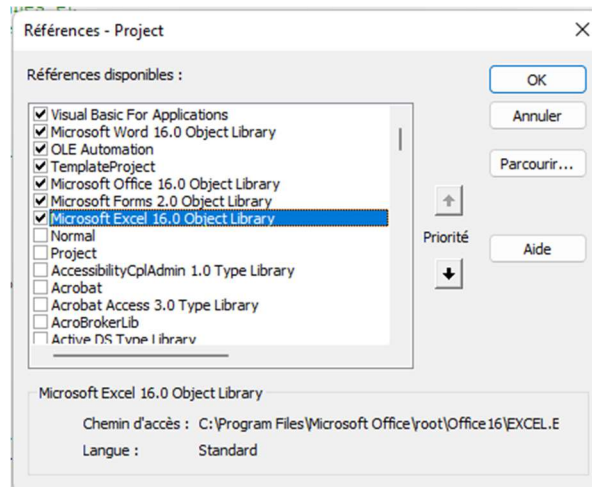


Puis cocher la librairie comme le montre la figure suivante :

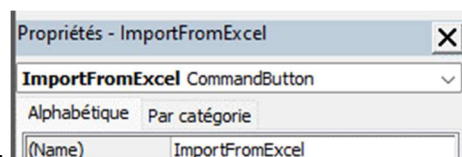


Ceci permettra à excel de comprendre les commandes qui parviendront depuis le Word dans la suite. Enfin, mettre à jour votre classeur excel.

13. Dans le document Word, allez dans la procédure associée à l'événement « click » associé au bouton « Importer depuis Excel »³. Procéder à l'activation de la librairie d'Objets Excel (comme fait précédemment) depuis word comme le montre la figure qui suit⁴:



14. Saisir le code suivant qui décrit par étapes les instructions à utiliser en langage VBA :



³ Ce bouton est nommé « ImportFromExcel » :

⁴ Cela permettra à Word de comprendre ce qui lui parviendra depuis Excel.

```

Private Sub ImportFromExcel_Click()

' Step 1 : déclarer les variables nécessaires
'-----
Dim WordAppli As Object ' Application Word
Dim ExcelAppli As Object ' Application Excel

Dim WordDocument As Object ' Document Word
Dim WordTable As Object ' Table cible
Dim ClasseurExcel As Object 'Fichier Excel

Dim FeuilleExcel As Worksheet ' Feuille Excel
Dim lastLigne As Long, lastColonne As Long 'Nb. Lignes et
'Colonnes remplies dans Excel
Dim i As Long, j As Long 'Compteurs d'incrément
Dim CheminVersClasseurExcel As String

' Step 2 : Initialiser l'objet Worksheet
'-----
' S'assurer que l'application Excel est ouverte
On Error Resume Next
Set ExcelAppli = GetObject(, "Excel.Application") 'test si ouverte
If Err.Number <> 0 Then
    Set ExcelAppli = CreateObject("Excel.Application") 'lancer Excel
End If
On Error GoTo 0

'Ouvrir le classeur Excel
' Préparer le chemin : Attention mettre le chemin selon votre machine
CheminVersClasseurExcel = "C:\Users\ENIGMA\Desktop\TP-ORAD\DataNotesCC_GX.xlsx"
Set ClasseurExcel = ExcelAppli.Workbooks.Open(CheminVersClasseurExcel)
' Visualiser Excel
ExcelAppli.Visible = True

'Accéder à la feuille de calcul
Set FeuilleExcel = ClasseurExcel.Sheets("DataGX") ' Préciser la feuille

' Step 3 : Récupérer les valeurs de lastLigne et lastColonne
'-----
lastLigne = FeuilleExcel.Cells(FeuilleExcel.Rows.Count, 1).End(xlUp).Row
lastColonne = FeuilleExcel.Cells(1, FeuilleExcel.Columns.Count).End(xlToLeft).Column

' Step 4 : Sélectionner la table à remplir
'-----
Set WordDocument = ThisDocument
Set WordTable = WordDocument.Tables(1) ' Table d'index 1

' Step 5 : Transfert des données vers la table cible
'-----
For i = 1 To lastLigne
    For j = 1 To lastColonne
        If i > WordTable.Rows.Count Then WordTable.Rows.Add
        If j > WordTable.Columns.Count Then WordTable.Columns.Add

        ' Copier la date vers la cellule correspondante
        WordTable.Cell(i, j).Range.Text = FeuilleExcel.Cells(i, j).Value
    Next j
Next i

End Sub

```

15. Tester le code. Le test devra vous afficher le résultat suivant :

Liste des Notes de Contrôle Continu

Matricule	Nom	Prénom	Note CC1	Note CC2
212100031515	ABDI	<u>Yousra</u>	10	12
222200031656	AYYAL	Hassan	13	12.75
202000034278	SELLAL	<u>Ahsen</u>	14.5	16
212100032623	DJABI	Islam	11	11
242400032387	DOUIB	Omar	7.5	11.75
222200032425	AYOUBI	<u>Ihssaan</u>	12.5	14
252500034568	DELLAL	<u>Hafidha</u>	8	7.25
202000034545	HENDOUCI	Nora	13	12
222200035689	BELKACEMI	Mourad	10	10

Ajouter Ligne ...

Importer Depuis Excel

16. Que se passe-t-il lorsque vous refaites le test ? Pourquoi ? Qu'en déduisez-vous ?
17. Comment procéder pour ajouter de nouveaux étudiants dans la liste sans passer par la table Word ?
18. Discuter les avantages de cette solution par rapport à la précédente.